

s for CI builds and Runner Fleet.

- can be used to ingest data from the CI build environment.

We may also use this database architecture to facilitate development of AI features in the future.

Our recent usability research on navigation and other areas suggests that the GitLab UI is overloaded with information and navigational elements.

This results from trying to add as much information as possible and attempting to place features in the most discoverable places.

Therefore, while developing these new observability features, we will rely on the jobs to be done research, and solution validation, to ensure that the features deliver the most value.

Runner Fleet

Metrics - MVC

What is the estimated wait time in queue for an instance runner?

The following customer problems should be solved when addressing this question. Most of them are quotes from our usability research

UI

- "There is no visibility for expected Runner queue wait times."

- "I got here looking for a view that makes it more obvious if I have a bottleneck on my specific runner."

Types of metrics

- "Is it possible to get metrics out of GitLab to check for the runners availability & pipeline wait times?"

Goal - we need the data to evaluate the data to determine if to scale up the Runner fleet so that there is no waiting times for developer's pipelines."

- "What is the estimated time in the Runner queue before a job can start?"

Interpreting metrics

- "What metrics for Runner queue performance should I look at and how do I interpret the metrics and take action?"

- "I want to be able to analyze data on Runner queue performance over time so that I can determine if the reports are from developers are really just rare cases regarding availability."

What is the estimated wait time in queue on a group runner?

What is the mean estimated wait time in queue for all instance runners?

What is the mean estimated wait time in queue for all group runners?

Which runners have failures in the past hour?

CI Insights

CI Insights is a page that would mostly expose data on pipelines and jobs duration, with a multitude of different filters, search and dynamic graphs. To read more on this, see this related sub-section.

Implementation

The current implementation plan is based on a Proof of Concept.

For an up to date status, see epic 10682.

Database selection

In FY23, ClickHouse was selected as GitLab standard datastore for features with big data and insert-heavy requirements. So we have chosen it for our CI analytics as well.

Scope of data

We're starting with the denormalized version of the cibuilds table in the main database, which will include fields from some other tables. For example, cirunners and cirunnermachines.

Immutability is a key constraint in ClickHouse, so we only use finished builds.

Developing behind feature flags

It's hard to fully test data ingestion and query performance in development/staging environments.

That's why we plan to deliver those features to production behind feature flags and test the performance on real data.

Feature flags for data ingestion and APIs will be separate.

Data ingestion

Every time a job finished, a record will be created in a new pcifinishedbuildchsync events table, which includes the buildid and a processed value.

A background worker loops through unprocessed pcifinishedbuildchsync events records and push the denormalized cibuilds information from Postgres to ClickHouse.

At some point we most likely will need to

parallelize this worker because of the number of processed builds.

This will be achieved by having the cron worker accept an argument determining the number of workers. The cron worker

will use that argument to queue the respective number of workers that will actually perform the

syncing to ClickHouse.

We will start with most recent builds and will not upload all historical data.

"Raw data", materialized views and queries

Ingested data will go to the "raw data" table in ClickHouse.

This table will use ReplacingMergeTree engine to deduplicate rows in case data ingestion mechanism accidentally submits the same batch twice.

Raw data can be used directly to execute queries, but most of the time we will create specialized materialized views

using AggregatingMergeTree engine.

This will allow us to read significantly less data when performing queries.

Limitations and open questions

The topics below require further investigation.

Efficient way of querying data for namespaces

We start with the PoC available only for administrators,
but very soon we will need to implement features on the group level.

We can't just put denormalized "path" in the source table because it can be changed when groups or projects are moved.

The simplest way of solving this is to always filter builds by projectid,
but this may be inefficient and require reading a significant portion of all data because ClickHouse stores data in big batches.

Keeping the database schema up to date

Right now we don't have any mechanism equivalent to migrations we use for PostgreSQL.
While developing our first features we will maintain database schema by hand and continue developing mechanisms for migrations.

Re-uploading data after changing the schema

If we need to modify database schema, old data maybe incomplete.
In that case we can simply truncate the ClickHouse tables and re-upload (part of) the data.

SECTION: engineering/architecture/design-
documents/ci_builds_runner_fleet_metrics/ci_insights.md

Summary

As part of the Fleet Metrics, we would like to have a section dedicated to CI insights to help

users monitor pipelines and summarize findings about pipelines speed, common job failures and more. It would eventually offer actionables to help users optimize and fix issues with their CI/CD.

Motivation

We have a page for CI/CD Analytics that contain some very basic analytics on pipelines. Most of this information relates to the total number of pipelines over time, which does not give any real value to customers: projects will always see an increase of pipelines number over time, so the total number of pipelines is of little consequence.

Because this page lacks real insights, it makes understanding pipelines slowdowns or failures hard to track and becomes a very manual task. We want to empower users to optimize their workflow in a centralized place to avoid all of the manual labor associated with either querying the API for data and then manually parsing it or navigating the UI through dozens of pages until the insights or action required can be found.

As we are going to process large quantities of data relating to a project pipelines, there is potential to eventually summarize findings with an AI tool to give insights into job failures, pipeline slowdowns and flaky specs. As AI has become a crucial part of our product roadmap and Verify lacks any promising lead in that area, this page could be the center of this new addition.

- Deliver a new Pipelines Analysis Dashboard page
- Have excellent data visualization to help digest information quickly
- Flexible querying to let users get the information they want
- Clear actionables based on information presented in the page
- Show some default information on landing like pipelines duration over time and slowest jobs
- Make the CI/CD Analytics more accessible, liked and remembered (AKA, more page views)

Non-Goals

We do not aim to improve the GitLab project's pipeline speed. This feature could help us achieve this, but it is not a direct objective of this blueprint.

We also are not aiming to have AI in the first iteration and should instead focus on making as much information available and digestible as possible.

Proposal

Revamp the page for CI/CD Analytics to include more meaningful data so that users can troubleshoot their pipelines with ease. Here is a list of the main improvements:

Overall statistics

The current "overall statistics" will become a one line header in a smaller font to keep this information available, but without taking as much visual space. For the pipelines chart, we

will replace it with a stacked bar plot where each stack of a bar represents a status and each bar is a unit (in days, a day, in month a month and in years, a year) so users can keep track of how many pipelines ran in that specific unit of time and what percent of these pipelines ended up in failing or succeeding.

Pipeline duration graph

A new pipeline duration graph that can be customized by type (MR pipelines, pipeline on a specific branch, etc), number of runs and status (success, failed, etc) and will replace the current Pipeline durations for the last 30 commits chart. The existing chart checks the latest 30 commits made on the repository with no filtering so the results presented are not very valuable.

We also add jobs that failed multiple times and jobs that are the slowest in the last x pipelines on master. All of this is to support the effort of allowing users to query their pipelines data to figure out what they need to improve on or what kind of problems they are facing with their CI/CD configuration.

Visibility

Add a link in the pipelines page to increase the visibility of this feature. We can add a new option with the Run pipeline primary button.

Master Broken

Add a "Is master broken?" quick option that scans the last x pipelines on the main branch and check for failed jobs. All jobs that failed multiple times will be listed in a table with the option to create an incident from that list.

Color scheme

Rethink our current color schemes for data visualization when it comes to pipelines statuses. We currently use the default visualization colors, but they don't actually match with that colors user have grown accustomed to for pipeline/jobs statuses. There is an opportunity here to help user better understand their data through more relevant color schemes and better visualization.

Routing

Change the routing from pipelines/charts to pipelines/analytics since charts is a really restrictive terminology when talking about data visualization. It also doesn't really convey what this page is, which is a way to get information, not just nice charts. Then we can also get rid of the query parameter for the tabs and instead support first-class routing.

Design and implementation details

New API for aggregated data

This feature depends on having a new set of data available to us that aggregates jobs and pipelines insights and make them available to the client.

We'll start by aggregating data from ClickHouse, and probably only for gitlab.com, as the MVC. We will aggregate the data on the backend on the fly. So far ClickHouse has been very capable of such things.

We won't store the aggregated data anywhere (we'll probably have the materialized views in ClickHouse, but nothing more complex). Then if the features get traction, we can explore ways to bring these features to environments without ClickHouse

This way we can move fast, test our ideas with real users, and get feedback.

Feature flag

To develop this new analytic page, we will gate the new page behind a feature flag ciinsights, and conditionally render the old or new analytics page. Potentially, we could even add the flag on the controller to decide which route to render: the new /analytic when the flag is one, and the old /charts when it isn't.

Add analytics on page view

Make sure that we can get information on how often this page is viewed. If we do not have it, then let's implement some to know how visible this page is. The changes to this section should make the view count go up and we want to track this as a measure of success.

Routing

We are planning to have new routes for the page and some redirects to set up. To read more about the routing proposal, see the related issue

Pipelines duration graph

We want a way for user to query data about pipelines with a lot of different criterias. Most notably, query for only pipelines with the scope finished or by status success or failed. There is also the possibility to scope this to a ref, so users could either test for the main branch or maybe even a branch that has introduced a CI/CD change. We want branch comparison for pipeline speed.

To get more accurate data, we want to increase the count of pipelines requested. In graphql, we have a limit of 100 items and we will probably get performance degradations quite quickly. We need to define how we could get more data set for more accurate data visualization.

Jobs insights

Currently, there is no way to query a single job across multiple pipelines and it prevent us from doing a query that would look like this:

```
graphql
query getJob($projectPath: ID!, $jobName: String!){
  project(fullPath:$projectPath){
    job(name: $jobName, last: 100){
      nodes{
```

```
    id
    duration
  }
}
}
```

We want a unified way to query information for job analytics. Roughly, this could translate to the following:

```
ruby
getjobs(projectid:, jobname: nil, stage: nil, stageindex: nil, etc)
>
[{:id: 1, :duration: 134, :status: 'failed'}, etc]

getjobsstatistics(projectid, jobname:, etc)
>
[{:timebucket: '2024-01-01:00:00:00', :avgduration: 234, :count: 123, :statusescount: {:success: 123, :failed: 45, :cancelled: 45}}]
```

Revamping our charts

Explore new color scheme and nicer look on our charts. Colaborate with UX to determine whether this is something we had on our mind or not and support any initiative to have nicer, more modern looking charts as our charts are quite forgettable.

Alternative Solutions

New page

We could create a brand new page and leave this section as it is. The pro would be that we could perhaps have a more prominent placement in the Navigation under Build, while the cons are that we'd have clear overlap with the section.

Pipeline analysis per pipeline

There was an experiment in the past to add performance insights per pipeline. The experiment was removed and deemed not viable. Some of the findings were that:

- Users did not interact with the page as much as thought and would not click on the button to view insights
- Users who did click on the button did not try to get more insights into a job.
- Users did not leave feedback in the issue.

This experiment reveals to us mostly that users who go on the pipeline graph page `pipelines/:id` are not trying to improve the performance of pipelines. Instead, it is most likely that this page is used to debug pipeline failures, which means that they are from the IC/developer persona, not the DevOps engineer trying to improve the workflow. By having this section in a

more "broad" area, we expect a much better adoption and more useful actionables.

Do nothing

We could leave this section untouched and not add any new form of analytics. The pro here would be the saved resources and time. The cons are that we currently have no way to help customers improve their CI/CD configurations speed except reading our documentation. This revamped section would also be a great gateway for AI features and help user iteration on their setup.

SECTION: engineering/architecture/design-documents/ci_build_speed/_index.md

{{< engineering/design-document-header >}}

Summary

GitLab CI is a Continuous Integration platform which is widely used to run a variety of jobs, builds, pipelines. It was integrated into GitLab in September 2015 and has become one of the most beloved CI/CD solutions.

With years we've added a lot of new features and code to the GitLab CI platform. In order to retain the "one of the most beloved solutions" status, we also need keep attention to making it fast, reliable and secure. This design doc is describing the path towards the former: making GitLab CI fast by improving CI build speed.

Goals

1. Establish a CI Speed Benchmark, used to compare GitLab CI to other platforms.
1. Build CI Benchmark Framework to measure the GitLab CI speed over the long term.
1. Describe next steps for improving GitLab CI Build Speed.

Proposal

CI Speed Benchmark

First, we plan to build a CI Speed Benchmark solution, that will allow us to run specific scenarios on various CI/CD platform and ingest results into our data warehouse.

This will make it possible to define a baseline of the CI Build Speed for many different scenarios and track the progress we, and other providers, are making over time.

The core part of this goal is to define a set of scenarios that will allow us to build a proxy metrics for build speed. For example, we could run following scenarios:

1. Time to first byte of build log for echo "Hello World" build.

1. Time to result to perform a CPU-intensive cryptographic operation.
1. Time to result to perform a memory-intensive for a given amount of bytes.
1. Time to result to build a Linux kernel.

The scenarios should be idempotent and deterministic.

In the first iteration, we will only focus on the total job execution time, and not go into detail e.g. comparing specific startup times.

CI Benchmark Framework

Once we define scenarios that we want to implement, we should build a CI Benchmark Framework. The framework will be used to run scenarios in a Continuous Integration environment, and to send the results back to our data warehouse, for analysis and comparison.

The main principles behind design choices for the framework, are:

1. Make it CI-platform agnostic. Can run on any Continuous Integration platform.
1. Do not depend on any specific technology that might not be available on some platforms.
1. Easy installation setup, not requiring many dependencies. Zero-dependency would be ideal.
1. Send results back to GitLab through an HTTP request, unless there is a better way.

Improve CI Build Speed

Once we can measure CI Build Speed, improving it can be possible. We will define the next steps for improving the speed once we have initial results.

SECTION: engineering/architecture/design-documents/ci_build_speed/benchmark.md

In order to understand how GitLab CI performs in terms of CI build speed, we plan to build CI Build Speed Benchmarking Framework.

Benchmark

In order to run the benchmark, we will:

1. Install the benchmarking tool.
1. Start the tool.
1. Runs scenarios.
1. Report results back to GitLab data warehouse.

In the first iteration, we will focus on measuring the speed of GitLab CI, GitHub Actions, and CircleCI.

Principles

There are a few design principles we should abide by:

1. Make it CI-platform agnostic. Can run on any Continuous Integration platform.
1. Do not depend on any specific technology that might not be available on some platforms.
1. Easy installation setup, not requiring many dependencies. Zero-dependency would be ideal.
1. Send results back to GitLab through an HTTP request, unless there is a better way.
1. Read as much data about the environment running a build and send details in the telemetry.

Benchmarking: Client Side

The benchmarking tool should be able to measure every step of CI build execution:

1. Time from build requested to scenario execution started.
1. Monotonic time to execute each of the steps of the scenario.
1. Thread time to execute each of the steps of the scenario.
1. Time required to report results back to GitLab.

Ideally the tool could collect this data in the Open Telemetry Tracing format.

Go-based tool

One of the solutions that could meet the requirements / principles listed above, could be a Go-based binary, which would be installed on different CI platform using wget / curl or in a different convenient way. The benefits of using the binary are:

1. Easy installation method, without the need to use containers.
1. Few external dependencies for a statically-linked binary.
1. Many libraries available, for tracing or HTTP / API integrations.
1. Multi-threaded execution mode that broadens benchmarking scope.
1. Expressive language that can make it easier to maintain the scenarios.

Benchmarking: Server Side

Pipelines scheduler

In order to run the benchmark a new build / pipeline / job will have to be started on a continuous integration platform under test. Some platforms support scheduled pipelines, but this could make it difficult to measure the build start-up time. On alternative to consider during the implementation is to start pipelines using API trigger endpoints. Most of the CI platforms support this way of running pipelines, and we could pass the start-up time / pipeline creation request time in an argument, that then will be consumed by the benchmarking tool, and forwarded to the data warehouse along with the build benchmark telemetry.

Data warehouse

The server side, that will receive benchmarking telemetry, will eventually need

to forward the data to a data warehouse, in which we will be able to visualize results, like Kibana or our Observability / Tracing tooling.

Before doing that, it could be advisable to persist the payload in object storage, just in case we need to migrate historical entries to a different data warehouse later on.

SECTION: engineering/architecture/design-documents/ci_data_decay/_index.md

{{< engineering/design-document-header >}}

Summary

GitLab CI/CD, integrated into GitLab in 2015, is a crucial yet resource-intensive component that has experienced exponential growth, surpassing 1 billion builds by 2021.

Despite its evolution, the CI/CD data storage architecture remains largely unchanged since 2012, posing scalability challenges due to the vast volume of data stored in PostgreSQL.

The proposed strategies involves:

1. Partitioning CI/CD data tables to efficiently support large scale of data and reduce the risk of database performance degradation.
1. Reducing the growth rate of metadata by being efficient in the data to store at any given stage of the pipeline lifecycle.
1. Archiving pipeline data to consistently move less accessed data to other storage solutions like object storage and enforce a different access pattern.
1. Introducing configurable data retention policies.

This architectural overhaul aims to enhance reliability, scalability, and performance while maintaining data accessibility and compliance.

Goals

This architectural overhaul aims to enhance reliability, scalability, and performance while maintaining data accessibility and compliance.

Challenges

There are more than two billion rows describing CI/CD builds in GitLab.com's database. This data represents a sizable portion of the whole data stored in PostgreSQL database running on GitLab.com.

This volume contributes to significant performance problems, development challenges and is often related to production incidents.

We also expect a significant growth in the number of builds executed on GitLab.com in the upcoming years.

Opportunity

CI/CD data is subject to time-decay

because, usually, pipelines that are a few months old are not frequently accessed or are even not relevant anymore. Restricting access to processing pipelines that are older than a few months might help us to move this data out of the primary database, to a different storage, that is more performant and cost effective.

It is already possible to prevent processing builds that have been archived.

When a build gets archived it will not be possible to retry it, but we still do keep all the processing metadata in the database, and it consumes resources that are scarce in the primary database.

To improve performance and make it easier to scale CI/CD data storage we might want to follow these three tracks described below.

<!-- markdownlint-disable MD029 -->

1. Partition CI/CD builds queuing database tables
2. Partition CI/CD pipelines database tables
3. Reduce the rate of builds metadata table growth

<!-- markdownlint-enable MD029 -->

Principles

All the three tracks we will use to implement CI/CD time decay pattern are associated with some challenges. As we progress with the implementation we will need to solve many problems and devise many implementation details to make this successful.

Below, we documented a few foundational principles to make it easier for everyone to understand the vision described in this architectural blueprint.

Removing pipeline data

While it might be tempting to remove old or archived data from our databases this should be avoided. It is usually not desired to permanently remove user data unless consent is given to do so. We can, however, move data

to a different data store, like object storage.

Archived data can still be needed sometimes (for example for compliance or auditing reasons). We want to be able to retrieve this data if needed, as long as permanent removal has not been requested or approved by a user.

Accessing pipeline data in the UI

Implementing CI/CD data time-decay through partitioning might be challenging when we still want to make it possible for users to access data stored in many partitions.

We want to retain simplicity of accessing pipeline data in the UI. It will require some backstage changes in how we reference pipeline data from other resources, but we don't want to make it more difficult for users to find their pipelines in the UI.

We may need to add "Archived" tab on the pipelines / builds list pages, but we should be able to avoid additional steps / clicks when someone wants to view pipeline status or builds associated with a merge request or a deployment.

We also may need to disable search in the "Archived" tab on pipelines / builds list pages.

Accessing pipeline data through the API

We accept the possible necessity of building a separate API endpoint / endpoints needed to access pipeline data through the API.

In the new API users might need to provide a time range in which the data has been created to search through their pipelines / builds. To make it efficient it might be necessary to restrict access to querying data residing in more than two partitions at once. We can do that by supporting time ranges spanning the duration that equals to the builds archival policy.

It is possible to still allow users to use the old API to access archived pipelines data, although a user provided partition identifier may be required.

Other strategies considered

Partition CI/CD builds queuing database tables

While working on the CI/CD Scale blueprint, we have introduced a new architecture for queuing CI/CD builds for execution.

This allowed us to significantly improve performance. We still consider the new solution as an intermediate mechanism, needed before we start working on the next iteration. The following iteration that should improve the architecture of builds queuing even more (it might require moving off the PostgreSQL fully or

partially).

In the meantime we want to ship another iteration, an intermediate step towards more flexible and reliable solution. We want to partition the new queuing tables, to reduce the impact on the database, to improve reliability and database health.

Partitioning of CI/CD queuing tables does not need to follow the policy defined for builds archival. Instead we should leverage a long-standing policy saying that builds created more 24 hours ago need to be removed from the queue. This business rule is present in the product since the inception of GitLab CI.

Epic: Partition CI/CD builds queuing database tables.

For more technical details about this topic see [pipeline data partitioning design](#).

Iterations

All three tracks can be worked on in parallel:

1. Reduce the rate of builds metadata table growth.
1. Partition CI/CD pipelines database tables.
1. Partition CI/CD queuing tables using list partitioning

Status

In progress.

Timeline

- 2021-01-21: Parent CI Scaling blueprint merge request created.
- 2021-04-26: CI Scaling blueprint approved and merged.
- 2021-09-10: CI/CD data time decay blueprint discussions started.
- 2022-01-07: CI/CD data time decay blueprint merged.
- 2022-02-01: Blueprint updated with new content and links to epics.
- 2022-02-08: Pipeline partitioning PoC merge request started.
- 2022-02-23: Pipeline partitioning PoC successful
- 2022-03-07: A way to attach an existing table as a partition found and proven.
- 2022-03-23: Pipeline partitioning design Google Doc (GitLab internal) started: <https://docs.google.com/document/d/1ARdoTZDy4qLGf6Z1GIHh83-stGZLpqsibjKrOXMgc>.
- 2022-03-29: Pipeline partitioning PoC concluded.
- 2022-04-15: Partitioned pipeline data associations PoC shipped.
- 2022-04-30: Additional benchmarking started to evaluate impact.
- 2022-06-31: Pipeline partitioning design document merge request merged.
- 2022-09-01: Engineering effort started to implement partitioning.
- 2022-11-01: The fastest growing CI table partitioned: cibuildsmetadata.
- 2023-06-30: The second largest table partitioned: cibuilds.
- 2023-12-12: cibuilds and cibuildsmetadata growth is stopped by writing data to new partitions.

- 2024-02-05: cipelinevariables is partitioned.
- 2024-03-26: cijobartifacts is partitioned.
- 2024-04-26: cistages is partitioned.

SECTION: engineering/architecture/design-documents/ci_data_decay/pipeline_archival.md

{{< engineering/design-document-header >}}

Problem to solve

SECTION: engineering/architecture/design-documents/ci_data_decay/pipeline_partitioning.md

{{< engineering/design-document-header >}}

Context

Even if we move CI/CD metadata to a different store, or reduce the rate of metadata growth in a different way, the problem of having billions of rows describing pipelines, builds and artifacts, remains. We still may need to keep reference to the metadata we might store in object storage and we still do need to be able to retrieve this information reliably in bulk (or search through it).

It means that by moving data to object storage we might not be able to reduce the number of rows in CI/CD tables. Moving data to object storage should help with reducing the data size, but not the quantity of entries describing this data. Because of this limitation, we still want to partition CI/CD data to reduce the impact on the database (indices size, auto-vacuum time and frequency).

Epic: Partition CI/CD pipelines database tables.

What problem are we trying to solve?

Our intent here is not to move this data out of our primary database elsewhere. What want to divide very large database tables, that store CI/CD data, into multiple smaller ones, using PostgreSQL partitioning features.

We want to partition the CI/CD dataset, because some of the database tables are extremely large, which might be challenging in terms of scaling single node reads, even after we ship the CI/CD database decomposition.

We want to reduce the risk of database performance degradation by transforming a few of the largest database tables into smaller ones using PostgreSQL declarative partitioning.

How are CI/CD data decomposition, partitioning, and time-decay related?

CI/CD decomposition is an extraction of a CI/CD database cluster out of the "main" database cluster, to make it possible to have a different primary database receiving writes. The main benefit is doubling the capacity for writes and data storage. The new database cluster will not have to serve reads / writes for non-CI/CD database tables, so this offers some additional capacity for reads too.

CI/CD partitioning is dividing large CI/CD database tables into smaller ones. This will improve reads capacity on every CI/CD database node, because it is much less expensive to read data from small tables, than from large multi-terabytes tables. We can add more CI/CD database replicas to better handle the increase in the number of SQL queries that are reading data, but we need partitioning to perform a single read more efficiently. Performance in other aspects will improve too, because PostgreSQL will be more efficient in maintaining multiple small tables than in maintaining a very large database table.

CI/CD time-decay allows us to benefit from the strong time-decay characteristics of pipeline data. It can be implemented in many different ways, but using partitioning to implement time-decay might be especially beneficial. When implementing a time decay we usually mark data as archived, and migrate it out of a database to a different place when data is no longer relevant or needed. Our dataset is extremely large (tens of terabytes), so moving such a high volume of data is challenging. When time-decay is implemented using partitioning, we can archive the entire partition (or set of partitions) by updating a single record in one of our database tables. It is one of the least expensive ways to implement time-decay patterns at a database level.

Why do we need to partition CI/CD data?

We need to partition CI/CD data because our database tables storing pipelines, builds, and artifacts are too large. The cibuilds database table size is currently around 2.5 TB with an index of around 1.4 GB. This is too much and violates our principle of 100 GB max size.

We also want to build alerting to notify us when this number is exceeded.

Large SQL tables increase index maintenance time, during which freshly deleted tuples cannot be cleaned by autovacuum. This highlights the need for small tables.

We will measure how much bloat we accumulate when (re)indexing huge tables. Based on this analysis,

we will be able to set up SLO (dead tuples / bloat), associated with (re)indexing.

We've seen numerous S1 and S2 database-related production environment

incidents, over the last couple of months, for example:

- S1: 2022-03-17 Increase in writes in cibuilds table
- S1: 2021-11-22 Excessive buffer read in replicas for cijobartifacts
- S2: 2022-04-12 Transactions detected that have been running for more than 10m
- S2: 2022-04-06 Database contention plausibly caused by excessive cibuilds reads
- S2: 2022-03-18 Unable to remove a foreign key on cibuilds
- S2: 2022-10-10 The queuingqueriesduration SLI apdex violating SLO

We have approximately 50 ci prefixed database tables, and some of them would benefit from partitioning.

A simple SQL query to get this data:

sql

```
WITH tables AS (SELECT tablename FROM informationschema.tables WHERE tablename LIKE 'ci%')
SELECT tablename,
    pgsizepretty(pgtotalrelationsize(quoteident(tablename))) AS totalsize,
    pgsizepretty(pgreferencesize(quoteident(tablename))) AS tablesizes,
    pgsizepretty(pgindexessize(quoteident(tablename))) AS indexsize,
    pgtotalrelationsize(quoteident(tablename)) AS totalsizebytes
FROM tables ORDER BY totalsizebytes DESC;
```

See data from March 2022:

Table name	Total size	Index size
cibuilds	3.5 TB	1 TB
cibuildsmetadata	1.8 TB	150 GB
cijobartifacts	600 GB	300 GB
cipipelines	400 GB	300 GB
cistages	200 GB	120 GB
cipelinevariables	100 GB	20 GB
(...around 40 more)		

Based on the table above, it is clear that there are tables with a lot of stored data.

While we have almost 50 CI/CD-related database tables, we are initially interested in partitioning only 6 of them. We can start by partitioning the most interesting tables in an iterative way, but we also should have a strategy for partitioning the remaining ones if needed. This document is an attempt to capture this strategy, describe as many details as possible, to share this knowledge among engineering teams.

How do we want to partition CI/CD data?

We want to partition the CI/CD tables in iterations. It might not be feasible to partition all of the 6 initial tables at once, so an iterative strategy

might be necessary. We also want to have a strategy for partitioning the remaining database tables when it becomes necessary.

It is also important to avoid large data migrations. We store almost 6 terabytes of data in the biggest CI/CD tables, in many different columns and indexes. Migrating this amount of data might be challenging and could cause instability in the production environment. Due to this concern, we've developed a way to attach an existing database table as a partition zero without downtime and excessive database locking, what has been demonstrated in one of the first proofs of concept.

This makes creation of a partitioned schema possible without a downtime (for example using a routing table `pcipipelines`), by attaching an existing `cipipelines` table as partition zero without exclusive locking. It will be possible to use the legacy table as usual, but we can create the next partition when needed and the `pcipipelines` table will be used for routing queries. To use the routing table we need to find a good partitioning key.

Our plan is to use logical partition IDs. We want to start with the `cipipelines` table and create a `partitionid` column with a `DEFAULT` value of 100 or 1000. Using a `DEFAULT` value avoids the challenge of backfilling this value for every row. Adding a `CHECK` constraint prior to attaching the first partition tells PostgreSQL that we've already ensured consistency and there is no need to check it while holding an exclusive table lock when attaching this table as a partition to the routing table (partitioned schema definition). We will increment this value every time we create a new partition for `pcipipelines`, and the partitioning strategy will be `LIST` partitioning.

We will also create a `partitionid` column in the other initial 6 database tables we want to iteratively partition. After a new pipeline is created, it will get a `partitionid` assigned, and all the related resources, like builds and artifacts, will share the same value. We want to add the `partitionid` column into all 6 problematic tables because we can avoid backfilling this data when we decide it is time to start partitioning them.

We want to partition CI/CD data iteratively. We plan to start with the `cibuildsmetadata` table, because this is the fastest growing table in the CI database and want to contain this rapid growth. This table has also the most simple access patterns - a row from it is being read when a build is exposed to a runner, and other access patterns are relatively simple too. Starting with `pcibuildsmetadata` will allow us to achieve tangible and quantifiable results earlier, and will become a new pattern that makes partitioning the largest table possible. We will partition builds metadata using the `LIST` partitioning strategy.

Once we have many partitions attached to `pcibuildsmetadata`, with many `partitionids` we will choose another CI table to partition next. In that case we might want to use `RANGE` partitioning in for that next table because `pcibuildsmetadata` will already have many physical partitions, and therefore many logical `partitionids` will be used at that time. For example,

if we choose cibuilds as the next partitioning candidate, after having partitioned pcibuildsmetadata, it will have many different values stored in cibuilds.partitionid. Using RANGE partitioning in that case might be easier.

Physical partitioning and logical partitioning will be separated, and a strategy will be determined when we implement physical partitioning for the respective database tables. Using RANGE partitioning works similarly to using LIST partitioning in database tables, but because we can guarantee continuity of partitionid values, using RANGE partitioning might be a better strategy.

Multi-project pipelines

Parent-child pipeline will always be part of the same partition because child pipelines are considered a resource of the parent pipeline. They can't be viewed individually in the project pipeline list page.

On the other hand, multi-project pipelines can be viewed in the pipeline list page. They can also be accessed from the pipeline graph as downstream/upstream links when created via the trigger token or the API using a job token. They can also be created from other pipelines by using trigger tokens, but in this case we don't store the source pipeline.

While partitioning cibuilds we need to update the foreign keys to the cisourcespipelines table:

plain

Foreign-key constraints:

```
"fkbe5624bf37" FOREIGN KEY (sourcejobid) REFERENCES cibuilds(id) ON DELETE CASCADE
"fk4e29af7d7" FOREIGN KEY (sourcepipelineid) REFERENCES cipipelines(id) ON DELETE CASCADE
"fke1bad85861" FOREIGN KEY (pipelineid) REFERENCES cipipelines(id) ON DELETE CASCADE
```

A cisourcespipelines record references two cipipelines rows (parent and the child). Our usual strategy has been to add a partitionid to the table, but if we do it here we will force all multi-project pipelines to be part of the same partition.

We should add two partitionid columns for this table, a partitionid and a sourcepartitionid:

plain

Foreign-key constraints:

```
"fkbe5624bf37" FOREIGN KEY (sourcejobid, sourcepartitionid) REFERENCES cibuilds(id,
sourcepartitionid) ON DELETE CASCADE
"fk4e29af7d7" FOREIGN KEY (sourcepipelineid, sourcepartitionid) REFERENCES cipipelines(id,
sourcepartitionid) ON DELETE CASCADE
"fke1bad85861" FOREIGN KEY (pipelineid, partitionid) REFERENCES cipipelines(id,
partitionid) ON DELETE CASCADE
```

This solution is the closest to a two way door decision because:

- We retain the ability to reference pipelines in different partitions.
- If we later decide that we want to force multi-project pipelines in the same partition we could add a constraint to validate that both columns have the same value.

Why do we want to use explicit logical partition ids?

Partitioning CI/CD data using a logical partitionid has several benefits. We could partition by a primary key, but this would introduce much more complexity and additional cognitive load required to understand how the data is being structured and stored in partitions.

CI/CD data is hierarchical data. Stages belong to pipelines, builds belong to stages, artifacts belong to builds (with rare exceptions). We are designing a partitioning strategy that reflects this hierarchy, to reduce the complexity and therefore cognitive load for contributors. With an explicit partitionid associated with a pipeline, we can cascade the partition ID number when trying to retrieve all resources associated with a pipeline. We know that for a pipeline 12345 with a partitionid of 102, we are always able to find associated resources in logical partitions with number 102 in other routing tables, and PostgreSQL will know in which partitions these records are being stored in for every table.

Another interesting benefit for using a single and incremental latest partitionid number, associated with pipelines, is that in theory we can cache it in Redis or in memory to avoid excessive reads from the database to find this number, though we might not need to do this.

The single and uniform partitionid value for pipeline data gives us more choices later on than primary-keys-based partitioning.

Altering partitioned tables

It will still be possible to run ALTER TABLE statements against partitioned tables, similarly to how the tables behaved before partitioning. When PostgreSQL runs an ALTER TABLE statement against a parent partitioned table, it acquires the same lock on all child partitions and updates each to keep them in sync. This differs from running ALTER TABLE on a non-partitioned table in a few key ways:

- PostgreSQL acquires ACCESS EXCLUSIVE locks against a larger number of tables, but not a larger amount of data, than it would were the table not partitioned. Each partition will be locked similarly to the parent table, and all will be updated in a single transaction.
- Lock duration will be increased based on the number of partitions involved. All ALTER TABLE statements executed on the GitLab database (other than VALIDATE CONSTRAINT) take small constant amounts of time per table modified. PostgreSQL will need to modify each partition in sequence, increasing the runtime of the lock. This time will still remain very small until there are many partitions involved.

- If thousands of partitions are involved in an ALTER TABLE, we will need to verify that the value of maxlockspertransaction is high enough to support all of the locks that need to be taken during the operation.

Splitting large partitions into smaller ones

We want to start with the initial partitionid number 100 (or higher, like 1000, depending on our calculations and estimations). We do not want to start from 1, because existing tables are also large already, and we might want to split them into smaller partitions. If we start with 100, we will be able to create partitions for partitionid of 1, 20, 45, and move existing records there by updating partitionid from 100 to a smaller number.

PostgreSQL will move these records into their respective partitions in a consistent way, provided that we do it in a transaction for all pipeline resources at the same time. If we ever decide to split large partitions into smaller ones (it's not yet clear if we will need to do this), we might be able to just use background migrations to update partition IDs, and PostgreSQL is smart enough to move rows between partitions on its own.

Naming conventions

A partitioned table is called a routing table and it will use the p prefix which should help us with building automated tooling for query analysis.

A table partition will be called partition and it can use the a physical partition ID as suffix, for example cibuilds101. Existing CI tables will become zero partitions of the new routing tables. Depending on the chosen partitioning strategy for a given table, it is possible to have many logical partitions per one physical partition.

Attaching first partition and acquiring locks

We learned when partitioning the first table that PostgreSQL requires an AccessExclusiveLock on the table and all of the other tables that it references through foreign keys. This can cause a deadlock if the migration tries to acquire the locks in a different order from the application business logic.

To solve this problem, we introduced a priority locking strategy to avoid further deadlock errors. This allows us to define the locking order and then try keep retrying aggressively until we acquire the locks or run out of retries. This process can take up to 40 minutes.

With this strategy, we successfully acquired a lock on cibuilds table after 15 retries during a low traffic period(after 00:00 UTC).

See an example of this strategy in our partition tooling).

Partitioning steps

The database partition tooling docs contain a list of steps to partition a table, but the steps are not enough for our iterative strategy. As our dataset continues to grow we want to take advantage of partitioning performance right away and not wait until all tables are partitioned. For example, after partitioning the `cibuildsmetadata` table we want to start writing and reading data to/from a new partition. This means that we will increase the `partitionid` value from 100, the default value, to 101. Now all of the new resources for the pipeline hierarchy will be persisted with `partitionid = 101`. We can continue following the database tooling instructions for the next table that will be partitioned, but we require a few extra steps:

- add `partitionid` column for the FK references with default value of 100 since the majority of records should have that value.
- change application logic to cascade the `partitionid` value
- correct `partitionid` values for recent records with a post deploy/background migration, similar to this:

```
sql
UPDATE cipipelinemetadata
  SET partitionid = cpipelines.partitionid
  FROM cpipelines
  WHERE cpipelines.id = cipipelinemetadata.pipelineid
  AND cpipelines.partitionid in (101, 102);
```

- change the foreign key definitions
- ...

Storing partitions metadata in the database

To build an efficient mechanism that will be responsible for creating new partitions, and to implement time decay we want to introduce a partitioning metadata table, called `cipartitions`. In that table we would store metadata about all the logical partitions, with many pipelines per partition. We may need to store a range of pipeline ids per logical partition. Using it we will be able to find the `partitionid` number for a given pipeline ID and we will also find information about which logical partitions are "active" or "archived", which will help us to implement a time-decay pattern using database declarative partitioning.

Doing that will also allow us to use a Unified Resource Identifier for partitioned resources, that will contain a pointer to a pipeline ID, we could then use to efficiently lookup a partition the resource is stored in. It might be important when a resource can be directly referenced by an URL, in UI or API. We could use an ID like `1e240-5ba0` for pipeline 123456, build 23456. Using a dash - can prevent an identifier from being highlighted and copied with a mouse double-click. If we want to avoid this problem, we can use any character of written representation that is not present in base-16 numeral system - any letter from g to z in Latin alphabet, for example x. In that

case an example of an URI would look like 1e240x5ba0. If we decide to update the primary identifier of a partitioned resource (today it is just a big integer) it is important to design a system that is resilient to migrating data between partitions, to avoid changing identifiers when rebalancing happens.

cipartitions table will store information about a partition identifier, pipeline ids range it is valid for and whether the partitions have been archived or not. Additional columns with timestamps may be helpful too.

Implementing a time-decay pattern using partitioning

We can use cipartitions to implement a time-decay pattern using declarative partitioning. By telling PostgreSQL which logical partitions are archived we can stop reading from these partitions using a SQL query like the one below.

```
sql
SELECT FROM cibuilds WHERE partitionid IN (
  SELECT id FROM cipartitions WHERE active = true
);
```

This query will make it possible to limit the number of partitions we will read from, and therefore will cut access to "archived" pipeline data, using our data retention policy for CI/CD data. Ideally we do not want to read from more than two partitions at once, so we need to align the automatic partitioning mechanisms with the time-decay policy. We will still need to implement new access patterns for the archived data, presumably through the API, but the cost of storing archived data in PostgreSQL will be reduced significantly this way.

There are some technical details here that are out of the scope of this description, but by using this strategy we can "archive" data, and make it much less expensive to reside in our PostgreSQL cluster by toggling a boolean column value.

Accessing partitioned data

It will be possible to access partitioned data whether it has been archived or not, in most places in GitLab. On a merge request page, we will always show pipeline details even if the merge request was created years ago. We can do that because cipartitions will be a lookup table associating a pipeline ID with its partitionid, and we will be able to find the partition that the pipeline data is stored in.

We will need to constrain access to searching through pipelines, builds, artifacts etc. Search cannot be done through all partitions, as it would not be efficient enough, hence we will need to find a better way of searching through archived pipelines data. It will be necessary to have different access patterns to access archived data in the UI and API.

There are a few challenges in enforcing usage of the partitionid

partitioning key in PostgreSQL. To make it easier to update our application to support this, we have designed a new queries analyzer in our proof of concept merge request. It helps to find queries that are not using the partitioning key.

In a separate proof of concept merge request and related issue we demonstrated that using the uniform partitionid makes it possible to extend Rails associations with an additional scope modifier so we can provide the partitioning key in the SQL query.

Using instance dependent associations, we can easily append a partitioning key to SQL queries that are supposed to retrieve associated pipeline resources, for example:

```
ruby
hasmany :builds, -> (pipeline) { where(partitionid: pipeline.partitionid) }
```

The problem with this approach is that it makes preloading much more difficult as instance dependent associations cannot be used with preloads:

```
plaintext
ArgumentError: The association scope 'builds' is instance dependent (the scope block takes an argument). Preloading instance dependent scopes is not supported.
```

Query analyzers

We implemented 2 query analyzers to detect queries that need to be fixed so that everything keeps working with partitioned tables:

- One analyzer to detect queries not going through a routing table.
- One analyzer to detect queries that use routing tables without specifying the partitionid in the WHERE clauses.

We started by enabling our first analyzer in test environment to detect existing broken queries. It is also enabled on production environment, but for a small subset of the traffic (0.1%) because of scalability concerns.

The second analyzer will be enabled in a future iteration.

Primary key

Primary key must include the partitioning key column to partition the table.

We first create a unique index including the (id, partitionid). Then, we drop the primary key constraint and use the new index created to set

the new primary key constraint.

ActiveRecord does not support composite primary keys, so we must force it to treat the id column as a primary key:

```
ruby
class Model < ApplicationRecord
  self.primarykey = 'id'
end
```

The application layer is now ignorant of the database structure and all of the existing queries from ActiveRecord continue to use the id column to access the data. There is some risk to this approach because it is possible to construct application code that results in duplicate models with the same id value, but on a different partitionid. To mitigate this risk we must ensure that all inserts use the database sequence to populate the id since they are guaranteed to allocate distinct values and rewrite the access patterns to include the partitionid value. Manually assigning the ids during inserts must be avoided.

Foreign keys

Foreign keys must reference columns that either are a primary key or form a unique constraint. We can define them using these strategies:

Between routing tables sharing partition ID

For relations that are part of the same pipeline hierarchy it is possible to share the partitionid column to define the foreign key constraint:

```
plaintext
pcipipelines:
- id
- partitionid
```

```
pcibuilds:
- id
- partitionid
- pipelineid
```

In this case, pcibuilds.partitionid indicates the partition for the build and also for the pipeline. We can add a FK on the routing table using:

```
sql
ALTER TABLE ONLY pcibuilds
  ADD CONSTRAINT fkonpipelineandpartition
  FOREIGN KEY (pipelineid, partitionid)
  REFERENCES pcipipelines(id, partitionid) ON DELETE CASCADE;
```

Between routing tables with different partition IDs

It's not possible to reuse the partitionid for all relations in the CI domain, so in this case we'll need to store the value as a different attribute. For example, when canceling redundant pipelines we store on the old pipeline row the ID of the new pipeline that cancelled it as autocanceledbyid:

plaintext

pcipipelines:

- id
- partitionid
- autocanceledbyid
- autocanceledbypartitionid

In this case we can't ensure that the canceling pipeline is part of the same hierarchy as the canceled pipelines, so we need an extra attribute to store its partition, autocanceledbypartitionid, and the FK becomes:

sql

```
ALTER TABLE ONLY pcipipelines
  ADD CONSTRAINT fkcancelredundantpipelines
  FOREIGN KEY (autocanceledbyid, autocanceledbypartitionid)
  REFERENCES pcipipelines(id, partitionid) ON DELETE SET NULL;
```

Between routing tables and regular tables

Not all of the tables in the CI domain will be partitioned, so we'll have routing tables that will reference non-partitioned tables, for example we reference externalpullrequests from cpipelines:

sql

```
FOREIGN KEY (externalpullrequestid)
REFERENCES externalpullrequests(id)
ON DELETE SET NULL
```

In this case we only need to move the FK definition from the partition level to the routing table so that new pipeline partitions may use it:

sql

```
ALTER TABLE pcipipelines
  ADD CONSTRAINT fkexternalrequest
  FOREIGN KEY (externalpullrequestid)
  REFERENCES externalpullrequests(id) ON DELETE SET NULL;
```

Between regular tables and routing tables

Most of the tables from the CI domain reference at least one table that will be turned into a routing tables, for example cipelinemessages references ciplines. These definitions will need to be updated to use the routing tables and for this they will need a partitionid column:

plaintext

pciplines:

- id
- partitionid

ciplinemessages:

- id
- pipelineid
- pipelinepartitionid

The foreign key can be defined by using:

sql

```
ALTER TABLE ciplinemessages ADD CONSTRAINT fkpipelinepartitioned
    FOREIGN KEY (pipelineid, pipelinepartitionid)
    REFERENCES pciplines(id, partitionid) ON DELETE CASCADE;
```

The old FK definition will need to be removed, otherwise new inserts in the ciplinemessages with pipeline IDs from non-zero partition will fail with reference errors.

Indexes

We learned that PostgreSQL does not allow to create a single index (unique or otherwise) across all partitions of a table.

One solution to solve this problem is to embed the partitioning key inside the uniqueness constraint.

This might mean prepending the partition ID in a hexadecimal format before the token itself and storing the concatenated string in a database. To do that we would need to reserve an appropriate number of leading bytes in a token to accommodate for the maximum number of partitions we may have in the future. It seems that reserving four characters, what would translate into 16-bits number in base-16, might be sufficient. The maximum number we can encode this way would be FFFF, what is 65535 in decimal.

This would provide a unique constraint per-partition which is sufficient for global uniqueness.

We have also designed a query analyzer that makes it possible to detect direct

usage of zero partitions, legacy tables that have been attached as first partitions to routing tables, to ensure that all queries are targeting partitioned schema or partitioned routing tables, like pcipipelines.

Why not partition using the project or namespace ID?

We do not want to partition using projectid or namespaceid because sharding and podding is a different problem to solve, on a different layer of the application. It doesn't solve the original problem statement of performance growing worse over time as we build up infrequently read data. We may want to introduce pods in the future, and that might become the primary mechanism of separating data based on the group or project the data is associated with.

In theory we could use either projectid or namespaceid as a second partitioning dimension, but this would add more complexity to a problem that is already very complex.

Partitioning builds queuing tables

We also want to partition our builds queuing tables. We currently have two: cipendingbuilds and cirunningbuilds. These tables are different from other CI/CD data tables, as there are business rules in our product that make all data stored in them invalid after 24 hours.

As a result, we will need to use a different strategy to partition those database tables, by removing partitions entirely after these are older than 24 hours, and always reading from two partitions through a routing table. The strategy to partition these tables is well understood, but requires a solid Ruby-based automation to manage the creation and deletion of these partitions. To achieve that we will collaborate with the Database team to adapt existing database partitioning tools to support CI/CD data partitioning.

Iterating to reduce the risk

This strategy should reduce the risk of implementing CI/CD partitioning to acceptable levels. We are also focusing on implementing partitioning for reading only from two partitions initially to make it possible to detach zero partitions in case of problems in our production environment. Every iteration phase, described below has a revert strategy and before shipping database changes we want to test them in our benchmarking environment.

The main way of reducing risk in case of this effort is iteration and making things reversible. Shipping changes, described in this document, in a safe and reliable way is our priority.

As we move forward with the implementation we will need to find even more ways to iterate on the design, support incremental rollouts and have better control over reverting changes in case of something going wrong. It is sometimes challenging to ship database schema changes iteratively, and even more

difficult to support incremental rollouts to the production environment. This can, however, be done, it just sometimes requires additional creativity, that we will certainly need here. Some examples of how this could look like:

Incremental rollout of partitioned schema

Once we introduce a first partitioned routing table (presumably pcipipelines) and attach its zero partition (cipipelines), we will need to start interacting with the new routing table, instead of a concrete partition zero. Usually we would override the database table the Ci::Pipeline Rails model would use with something like `self.table_name = 'pcipipelines'`. Unfortunately this approach might not support incremental rollout, because `self.table_name` will be read upon application boot up, and later we might be unable to revert this change without restarting the application.

One way of solving this might be introducing Ci::Partitioned::Pipeline model, that will inherit from Ci::Pipeline. In that model we would set `self.table_name` to pcipipeline and return its meta class from `Ci::Pipeline.partitioned` as a scope. This will allow us to use feature flags to route reads from cipipelines to pcipipelines with a simple revert strategy.

Incremental experimentation with partitioned reads

Another example would be related to the time when we decide to attach another partition. The goal of Phase 1 will be to have two partitions per partitioned schema / routing table, meaning that for pcipipelines we will have cipipelines attached as partition zero, and a new cipipelinesp1 partition created for new data. All reads from pcipipelines will also need to read data from the p1 partition and we should also iteratively experiment with reads targeting more than one partition, to evaluate performance and overhead of partitioning.

We can do that by moving old data to cipipelinesm1 (minus 1) partition iteratively. Perhaps we will create partitionid = 1 and move some really old pipelines there. We can then iteratively migrate data into m1 partition to measure the impact, performance and increase our confidence before creating a new partition p1 for new (still not created) data.

Iterations

We want to focus on Phase 1 iteration first. The goal and the main objective of this iteration is to partition the biggest 6 CI/CD database tables into 6 routing tables (partitioned schema) and 12 partitions. This will leave our Rails SQL queries mostly unchanged, but it will also make it possible to perform emergency detachment of "zero partitions" if there is a database performance degradation. This will cut users off their old data, but the application will remain up and running, which is a better alternative to application-wide outage.

1. Phase 0: Build CI/CD data partitioning strategy: Done. ·
1. Phase 1: Partition the 6 biggest CI/CD database tables.
 1. Create partitioned schemas for all 6 database tables.
 1. Design a way to cascade partitionid to all partitioned resources.
 1. Implement initial query analyzers validating that we target routing tables.
 1. Attach zero partitions to the partitioned database tables.
 1. Update the application to target routing tables and partitioned tables.
 1. Measure the performance and efficiency of this solution.

Revert strategy: Switch back to using concrete partitions instead of routing tables.

1. Phase 2: Add a partitioning key to add SQL queries targeting partitioned tables.
 1. Implement query analyzer to check if queries targeting partitioned tables are using proper partitioning keys.
 1. Modify existing queries to make sure that all of them are using a partitioning key as a filter.

Revert strategy: Use feature flags, query by query.

1. Phase 3: Build new partitioned data access patterns.
 1. Build a new API or extend an existing one to allow access to data stored in partitions that are supposed to be excluded based on the time-decay data retention policy.

Revert strategy: Feature flags.

1. Phase 4: Introduce time-decay mechanisms built on top of partitioning.
 1. Build time-decay policy mechanisms.
 1. Enable the time-decay strategy on GitLab.com.
1. Phase 5: Introduce mechanisms for creating partitions automatically.
 1. Make it possible to create partitions in an automatic way.
 1. Deliver the new architecture to self-managed instances.

The diagram below visualizes this plan on Gantt chart. The dates on the chart below ar